

回答例: Semantic Schedule Graph 版 Contract Model に慣れる

この文書は、`contract_model_schedule_semantic_graph.py` の演習課題集に対する回答例である。

注意

これは「唯一の正解」ではなく、設計意図を押さえた模範解答の一例である。
特に owner の置き方や product grammar の粒度には複数の妥当解がありうる。

Step 0: 全体像を言葉で整理する

回答 0-1

InputTemplate

ユーザーが少数の代表パラメータだけを入力するための入口である。
簡便入力のための層であり、source of truth ではない。
後からの局所修正や schedule dependency を完全には保持しない。

ContractForm

契約形態を保ったまま永続化・編集するための原本である。
legs / formulas / mechanisms / overrides / schedule graph を持つ。
このモジュールの中心であり、source of truth はここにある。

RuntimeState

観測やイベント処理の途中経過を保持する。
KO 済みか、target がどこまで積み上がったか、current notional がいくらか、などを持つ。
契約条項そのものではなく、契約条項を時系列に適用した結果である。

NormalizedView

異なる契約形態を比較・検索・集計するための派生表現である。

source of truth ではない。

same economics でも異なる form を潰さないため、比較時だけ使う。

回答 0-2

- ScheduleMeaning → 契約条項
- CashflowOverride → 契約条項
- RuntimeState.numeric_state → 途中経過
- InputTemplate → 入力補助

回答 0-3

NormalizedView を source of truth にしない理由は、

同じ経済効果でも契約形態の違いを保持したいからである。

たとえば Coupon Swap と FX Option Package は同じ economic meaning を持ちうるが、契約上・編集上・説明上は別 form として残した方が自然である。

Step 1: Semantic Schedule Graph の最小例を作る

回答 1-1

```
coupon_owner = ScheduleOwner(  
    owner_type=ScheduleOwnerType.LEG,  
    owner_id="coupon_leg",  
)  
  
payment_meaning = ScheduleMeaning(  
    roles=frozenset({DateRole.PAYMENT}),  
    owner=coupon_owner,  
)  
  
fixing_meaning = ScheduleMeaning(  
    roles=frozenset({DateRole.FIXING}),  
    owner=coupon_owner,  
)
```

回答 1-2

```
payment_node = ScheduleNode(  
    node_id=ScheduleNodeId("coupon_payment_dates"),  
    meaning=payment_meaning,  
    source=PatternScheduleSource(  
        pattern=SchedulePattern(  
            start_date=date(2026, 3, 31),  
            end_date=date(2027, 12, 31),  
            frequency="QUARTERLY",  
            end_of_month=True,  
        )  
    ),  
    description="Quarterly coupon payment dates",  
)  
  
fixing_node = ScheduleNode(  
    node_id=ScheduleNodeId("coupon_fixing_dates"),  
    meaning=fixing_meaning,  
    source=RelativeDateScheduleSource(  
        base_schedule_id=payment_node.node_id,  
        offset=-2,  
        unit=OffsetUnit.BUSINESS_DAYS,  
        business_day_convention=BusinessDayConvention.MODIFIED_FOLLOWING,  
    ),  
    description="Fixing = payment - 2 business days",  
)
```

回答 1-3

- root は payment_node
- derived は fixing_node
- fixing は payment に依存しており、依存方向は
payment_node -> fixing_node
である

つまり、payment dates がまず決まり、その結果から fixing dates が導かれる。

回答 1-4

```
dual_meaning = ScheduleMeaning(  
    roles=frozenset({DateRole.PAYMENT, DateRole.FIXING}),  
    owner=coupon_owner,  
)
```

role を単数で持つ設計より良い点は、

同じ日が複数の意味を持つことを自然に表せることである。

実務では payment date と fixing date が一致することがあり、その事実を素直に保持できる。

Step 2: ContractForm に schedule graph を載せる

回答 2-1

```
cp = CounterpartySpec(  
    book_party=PartyRef("BANK", "Bank"),  
    counterparty=PartyRef("CLIENT", "Client"),  
)  
usd_jpy = UnderlierRef("USDJPY", "FX")  
  
form = ContractForm(  
    form_id="FORM-SCHEDULE-GRAPH-MIN",  
    form_kind="DUMMY",  
    parties=cp.both(),  
    party_roles=(  
        PartyRoleAssignment("party_1", cp.book_party.party_id),  
        PartyRoleAssignment("party_2", cp.counterparty.party_id),  
    ),  
    references=(usd_jpy,),  
    transfers=(),  
    legs=(),  
    formulas=(),  
    mechanisms=(),  
    schedule_nodes=(payment_node, fixing_node),  
)
```

回答 2-2

```
form = replace(
    form,
    legs=(
        CouponLeg(
            component_id="coupon_leg",
            payer_party_id=cp.counterparty.party_id,
            receiver_party_id=cp.book_party.party_id,
            reference=usd_jpy,
            notional=SteppedDecimal(Decimal("1000000")),
            payment_schedule=ScheduleRef(payment_node.node_id),
            rate_formula_name="coupon_formula",
            currency=Currency.JPY,
        ),
    ),
    formulas=(
        FormulaBinding("coupon_formula", FixedRateFormula(SteppedDecimal(Decimal("0.08"
    ),
)
```

回答 2-3

`materialize()` を呼ぶと、`ContractForm` の中にある `ScheduleRef` が `resolver` によって解決され、concrete な `DateListSchedule` に置き換わる。
これにより、runtime や event timeline は rule graph を意識せず、通常の date list ベースで動ける。

Step 3: Coupon Swap の基本形を作る

回答 3-1

```
cp = CounterpartySpec(  
    book_party=PartyRef("BANK", "Bank"),  
    counterparty=PartyRef("CLIENT", "Client"),  
)  
usd_jpy = UnderlierRef("USDJPY", "FX")  
  
coupon_owner = ScheduleOwner(ScheduleOwnerType.LEG, "coupon_leg")  
  
payment_node = ScheduleNode(  
    node_id=ScheduleNodeId("coupon_payment_dates"),  
    meaning=ScheduleMeaning(  
        roles=frozenset({DateRole.PAYMENT}),  
        owner=coupon_owner,  
    ),  
    source=PatternScheduleSource(  
        pattern=SchedulePattern(  
            start_date=date(2026, 3, 31),  
            end_date=date(2027, 12, 31),  
            frequency="QUARTERLY",  
            end_of_month=True,  
        ),  
    ),  
)  
  
fixing_node = ScheduleNode(  
    node_id=ScheduleNodeId("coupon_fixing_dates"),  
    meaning=ScheduleMeaning(  
        roles=frozenset({DateRole.FIXING}),  
        owner=coupon_owner,  
    ),  
    source=RelativeDateScheduleSource(  
        base_schedule_id=payment_node.node_id,  
        offset=-2,  
        unit=OffsetUnit.BUSINESS_DAYS,  
    ),  
)
```



```

form = ContractForm(
    form_id="FORM-COUPON-SWAP-BASIC",
    form_kind="COUPON_SWAP",
    parties=cp.both(),
    party_roles=(
        PartyRoleAssignment("party_1", cp.book_party.party_id),
        PartyRoleAssignment("party_2", cp.counterparty.party_id),
    ),
    references=(usd_jpy,),
    transfers=(),
    legs=(
        CouponLeg(
            component_id="coupon_leg",
            payer_party_id=cp.counterparty.party_id,
            receiver_party_id=cp.book_party.party_id,
            reference=usd_jpy,
            notional=SteppedDecimal(Decimal("1000000")),
            payment_schedule=ScheduleRef(payment_node.node_id),
            rate_formula_name="coupon_formula",
            currency=Currency.JPY,
        ),
    ),
    formulas=(
        FormulaBinding("coupon_formula", FixedRateFormula(SteppedDecimal(Decimal("0.08"
    ),
    mechanisms=(),
    schedule_nodes=(payment_node, fixing_node),
)

```

回答 3-2

- payment rule → `payment_node.source = PatternScheduleSource(...)`
- fixing rule →
`fixing_node.source = RelativeDateScheduleSource(base=payment_node, offset=-2bd)`
- coupon stream → `CouponLeg`
- formula → `FixedRateFormula`
- parties → `CounterpartySpec + PartyRoleAssignment`

回答 3-3

payment node と fixing node の owner を同じ coupon_leg にするのは、
両者が同じ coupon stream の日付意味だからである。

fixing は coupon leg の rate determination に属する日付であり、payment と同じ leg に属すると見るのが自然である。

Step 4: Schedule patch と override を使う

回答 4-1

```
form_with_patch = replace(  
    form,  
    schedule_node_patches=(  
        ScheduleNodeIndexPatch(  
            node_id=payment_node.node_id,  
            occurrence_index=4,  
            new_date=date(2027, 4, 2),  
            reason="Special holiday arrangement",  
        ),  
    ),  
)
```

回答 4-2

payment node に patch を入れると、
payment schedule 自体が変更される。

もし fixing が payment に依存して materialize される設計なら、その影響が fixing 側にも波及する。

一方、fixing node に patch を入れると、payment は変わらず、fixing だけが局所的に変わる。
つまり、patch の付け先自体が意味を持つ。

回答 4-3

```
form_with_override = replace(
  form,
  overrides=(
    CashflowOverride(
      component_id="coupon_leg",
      payment_date=date(2026, 9, 30),
      field_name="coupon_rate",
      value=Decimal("0.12"),
      reason="Special 3rd coupon rate",
    ),
  ),
)
```

回答 4-4

- quarterly → monthly に変更 → rule 編集
- 第 5 回 payment を変更 → schedule patch
- 第 3 回 coupon 率だけ変更 → override

Step 5: KO を入れる

回答 5-1

```
ko_owner = ScheduleOwner(ScheduleOwnerType.MECHANISM, "ko_mech")

ko_obs_node = ScheduleNode(
    node_id=ScheduleNodeId("ko_observation_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.OBSERVATION}),
        owner=ko_owner,
    ),
    source=RelativeDateScheduleSource(
        base_schedule_id=payment_node.node_id,
        offset=-5,
        unit=OffsetUnit.BUSINESS_DAYS,
    ),
)

form_with_ko = replace(
    form,
    mechanisms=(
        KnockOutMechanism(
            component_id="ko_mech",
            predicate=BarrierPredicate(
                underlier=usd_jpy,
                direction=BarrierDirection.UP,
                level=Decimal("160"),
                observation_schedule=ScheduleRef(ko_obs_node.node_id),
            ),
            deactivate_components=("coupon_leg",),
        ),
    ),
    schedule_nodes=form.schedule_nodes + (ko_obs_node,),
)
```

回答 5-2

KO observation node の owner は `ko_mech` に置く方が自然だと思う。

理由は、この observation date は coupon leg 自体の payment/fixing ではなく、KO mechanism の trigger 判定のための日付だからである。

回答 5-3

`materialize()` 後には、KO mechanism の `BarrierPredicate.observation_schedule` は `ScheduleRef(...)` ではなく concrete な `DateListSchedule` になっている。

つまり runtime 側は graph を知らず、通常の observation date list として扱える。

Step 6: KO の効力スコープを component 分割で表す

回答 6-1

方針:

- `coupon_leg_first_4`
- `coupon_leg_post_4`
に分ける
- KO mechanism は `coupon_leg_post_4` だけ deactivate する

これにより、観察は最初から行いつつ、効力は 5 回目以降だけにできる。

回答 6-2

方針:

- `coupon_leg_odd`
- `coupon_leg_even`
に分ける
- KO mechanism は `coupon_leg_odd` だけ deactivate する

回答 6-3

mechanism に「odd only」「from 5th onward」などの特殊フラグを増やすと、mechanism がどんどん product-specific に肥大化する。

一方、component 分割なら

- trigger は mechanism
- effect scope は contract structure

と責務分離でき、条項との対応も見やすい。

Step 7: FX Option の form / meaning を理解する

回答 7-1

```
fx_tpl = FxOptionInputTemplate(  
    pair=FxPair(Currency.USD, Currency.JPY),  
    side=Side.BUY,  
    option_type=OptionType.CALL,  
    base_notional=Decimal("1000000"),  
    strike=Decimal("150.25"),  
    expiry_date=date(2026, 12, 18),  
    settlement_style=SettlementStyle.PHYSICAL,  
)  
form_fx = build_contract_form(fx_tpl)
```

回答 7-2

```
fx_leg = form_fx.legs[0]  
exchange_leg = fx_option_to_exchange_right(fx_leg)
```

この場合、USD call / JPY put なので、
meaning としては

- receive USD 1,000,000
- pay JPY 150,250,000

の交換権になる。

回答 7-3

FX option では市場慣行としては CALL / PUT や base / quote が自然だが、内部意味としては最終的に何を receive し何を pay するかが重要である。そのため、form-facing な表現と meaning-facing な表現を分けると、入力の自然さと内部意味の一意性を両立できる。

Step 8: FX Option Package を作る

回答 8-1

方針:

- long_call と short_put の 2 つの FxOptionExerciseLeg
- 同じ pair / strike / expiry
- ContractForm.form_kind = FX_OPTION_PACKAGE

回答 8-2

各 leg に対応する KnockInMechanism を用意し、

- eki_long_call
- eki_short_put

のように separate mechanism として表す。

回答 8-3

Barrier の owner を package 全体ではなく各 option leg / mechanism 側に寄せる理由は、ノックインの対象が package という抽象物ではなく、最終的には各 option の active/inactive だから

である。

この方が activation の責務が明確になる。

Step 9: Coupon Swap と FX Option Package を比較する

回答 9-1

ratio forward 的な payoff を表すとき、

- Coupon Swap では coupon stream として持つ
- FX Option Package では long call + short put の package として持つ

という 2 つの form がありうる。

回答 9-2

Coupon Swap

- form の違い: coupon leg の列
- schedule の自然さ: payment/fixing/observation が自然
- KO/KI の自然さ: KO の効力スコープを leg 分割で表しやすい
- internal meaning: coupon stream 的

FX Option Package

- form の違い: option legs の束
- schedule の自然さ: expiry / settlement / premium が自然
- KO/KI の自然さ: EKI/EKO を option activation として持ちやすい
- internal meaning: exchange-right に展開しやすい

回答 9-3

same economics でも same form に漬さない方がよい理由は、
契約形態の違いが入力・編集・説明・法的解釈に影響するからである。
比較したいときだけ normalized に漬せばよい。

Step 10: MtM Notional CCS を見る

回答 10-1

- schedule: いつ coupon / reset / principal exchange が起きるか
- mechanism: reset 時に何を更新するか
- runtime state: current notional がいくらになったか

回答 10-2

- payment of pay_leg $\rightarrow$ owner = LEG / pay_leg
- payment of receive_leg $\rightarrow$ owner = LEG / receive_leg
- reset of mtm_reset_mech $\rightarrow$ owner = MECHANISM / mtm_reset_mech
- principal exchange of form $\rightarrow$ owner = FORM / form_id

回答 10-3

current notional の更新は「いつ起きるか」ではなく「起きたとき何をするか」の問題なので、
schedule graph ではなく mechanism + runtime state の責務である。
schedule graph は reset date を与えるだけでよい。

Step 11: 自分で小さな grammar を設計する

回答 11-1

Coupon Swap grammar

- 必須 references: underlier
- 必須 schedule meanings: payment, fixing
- 許される legs: coupon leg
- 許される formulas: fixed / floating / digital-like coupon formulas
- 許される mechanisms: KO, KI, step-up, target
- 許される patch / override: payment/fixing patch, coupon rate override

回答 11-2

FX Option Package grammar

- 必須 references: fx pair
- 必須 legs: long call / short put などの FX option legs
- premium: transfer として持てる
- exercise / barrier: mechanism として持てる
- form / meaning 分離: form-facing は `FxOptionExerciseLeg`、meaning-facing は `FxExchangeRightLeg`

回答 11-3

product grammar は単なる UI 入力定義ではなく、
どの product kind でどの構成要素の組み合わせが許されるかを規定する authoring schema である。
したがって source of truth になる ContractForm の shape に深く関わる。

Step 12: まとめの総合課題

回答 12-1

Coupon Swap の例では、

- payment は `PatternScheduleSource`
- fixing は payment からの `RelativeDateScheduleSource`
- KO observation も payment からの relative node
- KO の効力対象は `coupon_leg_post_4` に限定
- 第 5 回 payment は `ScheduleNodeIndexPatch`
- 第 3 回 coupon rate は `CashflowOverride`

と切り分けるのが自然である。

回答 12-2

FX Option Package の例では、

- long call / short put の leg を package として持つ
- premium は transfer
- 各 option に EKI を `KnockInMechanism` として付与
- 必要なら internal meaning として `FxExchangeRightLeg` に展開

とするのが自然である。

回答 12-3

1. `ScheduleMeaning` を持つ利点
→ 日付の意味を node id ではなくドメイン構造として持てる
2. role を複数集合にする利点
→ Payment かつ Fixing のような複合的意味を自然に表せる
3. `ScheduleRef` を使う利点
→ component と node を疎結合にし、direct object cycle を避けられる
4. `materialize()` を分ける利点
→ contract substance と runtime artifact を分離できる

5. Coupon Swap と FX Option Package を両方やる意味

→ same economics でも異なる form を保持する重要性を理解できる

回答例 v2 追補: AccrualCouponLeg 前提の Coupon Swap

この文書は、SEMANTIC_SCHEDULE_GRAPH_ANSWERS.md のうち

Coupon Swap まわりを **AccrualCouponLeg 前提** で置き換えるための追補である。

前の回答は、Coupon Swap というより「coupon stream の最小例」に近かった。

ここでは、Coupon Swap をより自然な形で書き直す。

1. Step 3-1 の差し替え回答

問題

次の条件の Coupon Swap を作ってください。

- quarterly coupon payment
- fixing = payment - 2 business days
- coupon は JPY 支払い
- underlying は USDJPY
- coupon formula は fixed 8%

修正版回答

この問いだけだと receive leg が明示されていないが、

Coupon Swap として自然にするため、ここでは **2-leg 構造** にして回答する。

- pay leg: fixed 8%
- receive leg: fixed 8% (最小構造の都合で同率にしてよい)
- 両 leg とも payment / fixing / accrual を持つ

```

from datetime import date
from decimal import Decimal

from contract_model_schedule_semantic_graph_accrual_coupon import *

cp = CounterpartySpec(
    book_party=PartyRef("BANK", "Bank"),
    counterparty=PartyRef("CLIENT", "Client"),
)
usd_jpy = UnderlierRef("USDJPY", "FX")

# -----
# PAY LEG schedule meanings
# -----

pay_owner = ScheduleOwner(ScheduleOwnerType.LEG, "pay_coupon_leg")

pay_payment_node = ScheduleNode(
    node_id=ScheduleNodeId("pay_payment_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.PAYMENT}),
        owner=pay_owner,
    ),
    source=PatternScheduleSource(
        pattern=SchedulePattern(
            start_date=date(2026, 3, 31),
            end_date=date(2026, 12, 31),
            frequency="QUARTERLY",
            end_of_month=True,
        )
    ),
)

pay_fixing_node = ScheduleNode(
    node_id=ScheduleNodeId("pay_fixing_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.FIXING}),
        owner=pay_owner,
    ),
    source=RelativeDateScheduleSource(
        base_schedule_id=pay_payment_node.node_id,
        offset=-2,
        unit=OffsetUnit.BUSINESS_DAYS,
    )
)

```

```

    ),
)

pay_accrual_start_node = ScheduleNode(
    node_id=ScheduleNodeId("pay_accrual_start_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.ACCRUAL_START}),
        owner=pay_owner,
    ),
    source=ExplicitDateScheduleSource(
        schedule=DateListSchedule((
            date(2025, 12, 31),
            date(2026, 3, 31),
            date(2026, 6, 30),
            date(2026, 9, 30),
        ))
    ),
)

pay_accrual_end_node = ScheduleNode(
    node_id=ScheduleNodeId("pay_accrual_end_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.ACCRUAL_END}),
        owner=pay_owner,
    ),
    source=ExplicitDateScheduleSource(
        schedule=DateListSchedule((
            date(2026, 3, 31),
            date(2026, 6, 30),
            date(2026, 9, 30),
            date(2026, 12, 31),
        ))
    ),
)

# -----
# RECEIVE LEG schedule meanings
# -----

receive_owner = ScheduleOwner(ScheduleOwnerType.LEG, "receive_coupon_leg")

receive_payment_node = ScheduleNode(
    node_id=ScheduleNodeId("receive_payment_dates"),

```

```

meaning=ScheduleMeaning(
    roles=frozenset({DateRole.PAYMENT}),
    owner=receive_owner,
),
source=PatternScheduleSource(
    pattern=SchedulePattern(
        start_date=date(2026, 3, 31),
        end_date=date(2026, 12, 31),
        frequency="QUARTERLY",
        end_of_month=True,
    )
),
)

```

```

receive_fixing_node = ScheduleNode(
    node_id=ScheduleNodeId("receive_fixing_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.FIXING}),
        owner=receive_owner,
    ),
    source=RelativeDateScheduleSource(
        base_schedule_id=receive_payment_node.node_id,
        offset=-2,
        unit=OffsetUnit.BUSINESS_DAYS,
    ),
)

```

```

receive_accrual_start_node = ScheduleNode(
    node_id=ScheduleNodeId("receive_accrual_start_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.ACCRUAL_START}),
        owner=receive_owner,
    ),
    source=ExplicitDateScheduleSource(
        schedule=DateListSchedule((
            date(2025, 12, 31),
            date(2026, 3, 31),
            date(2026, 6, 30),
            date(2026, 9, 30),
        ))
    ),
)

```

```

receive_accrual_end_node = ScheduleNode(
    node_id=ScheduleNodeId("receive_accrual_end_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.ACCRUAL_END}),
        owner=receive_owner,
    ),
    source=ExplicitDateScheduleSource(
        schedule=DateListSchedule((
            date(2026, 3, 31),
            date(2026, 6, 30),
            date(2026, 9, 30),
            date(2026, 12, 31),
        ))
    ),
)

```

```

form = ContractForm(
    form_id="FORM-COUPON-SWAP-STEP3-V2",
    form_kind="COUPON_SWAP",
    parties=cp.both(),
    party_roles=(
        PartyRoleAssignment("payer", cp.counterparty.party_id),
        PartyRoleAssignment("receiver", cp.book_party.party_id),
    ),
    references=(usd_jpy,),
    transfers=(),
    legs=(
        AccrualCouponLeg(
            component_id="pay_coupon_leg",
            payer_party_id=cp.counterparty.party_id,
            receiver_party_id=cp.book_party.party_id,
            reference=usd_jpy,
            notional=SteppedDecimal(Decimal("1000000")),
            payment_schedule=ScheduleRef(pay_payment_node.node_id),
            accrual_start_schedule=ScheduleRef(pay_accrual_start_node.node_id),
            accrual_end_schedule=ScheduleRef(pay_accrual_end_node.node_id),
            fixing_schedule=ScheduleRef(pay_fixing_node.node_id),
            rate_formula_name="pay_rate_formula",
            currency=Currency.JPY,
            day_count=DayCount.ACT_365F,
        ),
        AccrualCouponLeg(
            component_id="receive_coupon_leg",

```



```

payer_party_id=cp.book_party.party_id,
receiver_party_id=cp.counterparty.party_id,
reference=usd_jpy,
notional=SteppedDecimal(Decimal("1000000")),
payment_schedule=ScheduleRef(receive_payment_node.node_id),
accrual_start_schedule=ScheduleRef(receive_accrual_start_node.node_id),
accrual_end_schedule=ScheduleRef(receive_accrual_end_node.node_id),
fixing_schedule=ScheduleRef(receive_fixing_node.node_id),
rate_formula_name="receive_rate_formula",
currency=Currency.JPY,
day_count=DayCount.ACT_365F,
    ),
),
formulas=(
    FormulaBinding("pay_rate_formula", FixedRateFormula(SteppedDecimal(Decimal("0.0
    FormulaBinding("receive_rate_formula", FixedRateFormula(SteppedDecimal(Decimal(
),
mechanisms=(),
schedule_nodes=(
    pay_payment_node,
    pay_fixing_node,
    pay_accrual_start_node,
    pay_accrual_end_node,
    receive_payment_node,
    receive_fixing_node,
    receive_accrual_start_node,
    receive_accrual_end_node,
),
)

```

説明

前の回答では CouponLeg 1 本しかなく、swap として弱かった。

ここでは AccrualCouponLeg を 2 本使うことで、

- swap らしい 2-leg 構造
- payment / fixing / accrual period
- coupon determination に必要な情報

を leg 側に持たせている。

2. Step 3-2 の差し替え回答

契約条項とオブジェクトの対応

条項

Each leg pays quarterly coupons.

- `pay_payment_node / receive_payment_node`
- `PatternScheduleSource(..., frequency="QUARTERLY")`

条項

Each leg has fixing two business days prior to payment.

- `pay_fixing_node / receive_fixing_node`
- `RelativeDateScheduleSource(base=payment_node, offset=-2bd)`

条項

Each coupon accrues over its own accrual period.

- `pay_accrual_start_node / pay_accrual_end_node`
- `receive_accrual_start_node / receive_accrual_end_node`

条項

Coupon leg structure

- `AccrualCouponLeg`

条項

Coupon rate rule

- `FormulaBinding("pay_rate_formula", ...)`
- `FormulaBinding("receive_rate_formula", ...)`

3. Step 3-3 の差し替え回答

問題

payment node と fixing node の owner を同じ leg にする理由を説明してください。

回答

payment node と fixing node の owner を同じ leg にする理由は、
どちらもその leg の coupon determination / settlement に属する schedule だからである。

さらに本格的には、

- PAYMENT
- FIXING
- ACCRUAL_START
- ACCRUAL_END

は 모두同じ AccrualCouponLeg の内部構造として理解するのが自然である。
したがって、semantic schedule graph では同じ leg owner を共有するのが筋が良い。

4. Step 5 の差し替え回答

問題

Coupon Swap に KO observation = payment - 5bd を追加し、KO hit で coupon leg を deactivate してください。

修正版回答

Coupon Swap に KO を入れるなら、「どの leg に効くか」を明示した方が自然である。
ここでは **receive leg のみ KO 対象** とする。

```

ko_owner = ScheduleOwner(ScheduleOwnerType.MECHANISM, "ko_mech")

ko_obs_node = ScheduleNode(
    node_id=ScheduleNodeId("ko_observation_dates"),
    meaning=ScheduleMeaning(
        roles=frozenset({DateRole.OBSERVATION}),
        owner=ko_owner,
    ),
    source=RelativeDateScheduleSource(
        base_schedule_id=receive_payment_node.node_id,
        offset=-5,
        unit=OffsetUnit.BUSINESS_DAYS,
    ),
)

form_with_ko = replace(
    form,
    mechanisms=(
        KnockOutMechanism(
            component_id="ko_mech",
            predicate=BarrierPredicate(
                underlier=usd_jpy,
                direction=BarrierDirection.UP,
                level=Decimal("160"),
                observation_schedule=ScheduleRef(ko_obs_node.node_id),
            ),
            deactivate_components=("receive_coupon_leg",),
        ),
    ),
    schedule_nodes=form.schedule_nodes + (ko_obs_node,),
)

```

説明

前の回答では KO が coupon stream 1 本に直接効く形で、Coupon Swap としては弱かった。
ここでは

- observation は mechanism owner に属する node
- KO は receive leg にだけ効く

と明示している。

5. Step 6 の差し替え回答

問題

KO の効力対象を 5 回目以降 / 奇数回のみにしてください。

回答方針

この場合も、対象となるのは `AccrualCouponLeg` 側である。
したがって、`CouponLeg` ではなく

- `receive_coupon_leg_first_4`
- `receive_coupon_leg_post_4`

あるいは

- `receive_coupon_leg_odd`
- `receive_coupon_leg_even`

のように、**`AccrualCouponLeg`** を分割して表すのが自然である。

6. 「各クーポンは個別に決まる」の整理

回答

Coupon Swap において「各クーポンは個別に決まる」とは、

- 各期で `fixing date` が違う
- 各期で `accrual period` が違う
- 各期で `accrual factor` が違う
- 必要なら `override` も違う

という意味である。

これを表すのに、必ずしも各期ごと別 formula を持つ必要はない。
むしろ自然なのは、

- leg は `AccrualCouponLeg`
- schedule graph が各期の fixing / accrual period を与える
- formula は共通ルール
- 実現値は runtime / valuation で決まる

という責務分離である。

7. まとめ

今回の差し替えで重要なのは次の点である。

- Coupon Swap の中心 leg は `CouponLeg` ではなく **`AccrualCouponLeg`**
- coupon determination の中心情報
 - payment
 - accrual start
 - accrual end
 - fixingを leg 側に持つ
- KO の対象 leg を明示する
- 「各期で個別に決まる」は schedule graph + runtime の責務分離で表す

つまり、前の回答の問題は

Coupon Swap を coupon stream の最小例で済ませていたことにあった。

この v2 では、それを `AccrualCouponLeg` 前提で修正している。